

uma variável **float**, mas não o contrário. Não permitir metade das possíveis atribuições de modo misto é uma maneira simples, mas efetiva, de aumentar a confiabilidade de Java e C# em relação à C e C++.

Evidentemente, nas linguagens funcionais, em que as atribuições são usadas apenas para dar nomes a valores, não existe nenhuma atribuição de modo misto.

RESUMO

Expressões são compostas de constantes, variáveis, parênteses, chamadas a funções e operadores. Sentenças de atribuição incluem variáveis-alvo, operadores de atribuição e expressões.

A semântica de uma expressão é determinada, em grande parte, pela ordem de avaliação dos operadores. As regras de associatividade e de precedência para operadores em expressões de uma linguagem determinam a ordem de avaliação dos operadores nessas expressões. A ordem de avaliação dos operandos é importante se efeitos colaterais funcionais são possíveis. Conversões de tipo podem ser de alargamento ou de estreitamento. Algumas conversões de estreitamento produzem valores errôneos. Conversões de tipo implícitas, ou coerções, em expressões são comuns. Apesar disso, eliminam alguns dos benefícios da verificação de tipos, diminuindo a confiabilidade.

Sentenças de atribuição têm aparecido em uma ampla variedade de formas, incluindo alvos condicionais, operadores de atribuição e atribuições de listas.

QUESTÕES DE REVISÃO

1. Defina *precedência de operador* e *associatividade de operador*.
2. O que é um operador ternário?
3. O que é um operador pré-fixado?
4. Que operador normalmente tem associatividade à direita?
5. O que é um operador não associativo?
6. Que regras de associatividade são usadas por APL?
7. Qual é a diferença entre a maneira pela qual os operadores são implementados em C++ e Ruby?
8. Defina *efeito colateral funcional*.
9. O que é uma coerção?
10. O que é uma expressão condicional?
11. O que é um operador sobrecarregado?
12. Defina *conversões de alargamento* e *de estreitamento*.
13. Em JavaScript, qual é a diferença entre `==` e `===`?

14. O que é uma expressão de modo misto?
15. O que é transparência referencial?
16. Quais são as vantagens da transparência referencial?
17. Como a ordem de avaliação dos operandos interage com os efeitos colaterais funcionais?
18. O que é a avaliação em curto-circuito?
19. Cite uma linguagem que sempre faz avaliação em curto-circuito de expressões booleanas. Cite uma que nunca faz isso.
20. Como C oferece suporte para expressões relacionais e booleanas?
21. Qual é o propósito de um operador de atribuição composto?
22. Qual é a associatividade dos operadores aritméticos unários de C?
23. Qual é uma possível desvantagem de tratar o operador de atribuição como se ele fosse um operador aritmético?
24. Que duas linguagens incluem atribuições múltiplas?
25. Que atribuições de modo misto são permitidas em Java?
26. Que atribuições de modo misto são permitidas em ML?
27. O que é um *cast*?

PROBLEMAS

1. Quando você poderia desejar que o compilador ignorasse diferenças de tipos em uma expressão?
2. Argumente a favor e contra permitir expressões aritméticas de modo misto.
3. Você acha que a eliminação de operadores sobrecarregados em sua linguagem favorita seria benéfica? Justifique sua resposta.
4. Seria uma boa ideia eliminar todas as regras de precedência de operadores e exigir parênteses para mostrar a precedência desejada em expressões? Justifique sua resposta.
5. As operações de atribuição de C (por exemplo, +=) deveriam ser incluídas em outras linguagens (que ainda não as têm)? Justifique sua resposta.
6. As formas de atribuição de um único operando de C (por exemplo, ++count) deveriam ser incluídas em outras linguagens (que ainda não as têm)? Justifique sua resposta.
7. Descreva uma situação na qual o operador de adição em uma linguagem de programação não seria comutativo.

8. Descreva uma situação na qual o operador de adição em uma linguagem de programação não seria associativo.
9. Suponha as seguintes regras de associatividade e de precedência para expressões:

<i>Precedência</i>	<i>Mais alta</i>	* , / , not + , - , & , mod - (unário) = , /= , < , <= , >= , > and or , xor
<i>Associatividade</i>	<i>Mais baixa</i> <i>Esquerda para a direita</i>	

Mostre a ordem de avaliação das seguintes expressões por meio do uso de parênteses em todas as subexpressões e da colocação de um expoente no parêntese direito para indicar a ordem. Por exemplo, para a expressão

$a + b * c + d$

a ordem de avaliação seria representada como

$((a + (b * c)^1)^2 + d)^3$

- a. $a * b - 1 + c$
 - b. $a * (b - 1) / c \text{ mod } d$
 - c. $(a - b) / c \& (d * e / a - 3)$
 - d. $-a \text{ or } c = d \text{ and } e$
 - e. $a > b \text{ xor } c \text{ or } d \leq 17$
 - f. $-a + b$
10. Mostre a ordem de avaliação das expressões do Problema 9, supondo que não existem regras de precedência e que todos os operadores são associativos da direita para a esquerda.
 11. Escreva uma descrição em BNF para as regras de precedência e de associatividade definidas para as expressões no Problema 9. Suponha que os únicos operandos são os nomes a, b, c, d e e .
 12. Usando a gramática do Problema 11, desenhe árvores de análise sintática para as expressões do Problema 9.
 13. Seja a função `fun` definida como

```
int fun(int*k) {
    *k += 4;
    return 3 * (*k) - 1;
}
```

Suponha que `fun` seja usada em um programa, como:

```

void main() {
    int i = 10, j = 10, sum1, sum2;
    sum1 = (i / 2) + fun(&i);
    sum2 = fun(&j) + (j / 2);
}

```

Quais são os valores de `sum1` e `sum2`

- a. se os operandos na expressão forem avaliados da esquerda para a direita?
 - b. se os operandos na expressão forem avaliados da direita para a esquerda?
14. Qual seu principal argumento contra as (ou a favor das) regras de precedência de operadores de APL?
 15. Explique por que é difícil eliminar efeitos colaterais funcionais em C.
 16. Para alguma linguagem de sua escolha, crie uma lista de símbolos de operadores que poderiam ser usados para eliminar todas as sobrecargas de operadores.
 17. Determine se as conversões de tipo explícitas de estreitamento em duas linguagens que você conhece fornecem mensagens de erro quando um valor convertido perde sua utilidade.
 18. Deveria ser permitido a um compilador com otimização para C e C++ modificar a ordem das subexpressões em uma expressão booleana? Justifique sua resposta.
 19. Considere o seguinte programa em C:

```

int fun(int *i) {
    *i += 5;
    return 4;
}
void main() {
    int x = 3;
    x = x + fun(&x);
}

```

Qual é o valor de `x` após a sentença de atribuição em `main`, supondo que

- a. os operandos são avaliados da esquerda para a direita.
 - b. os operandos são avaliados da direita para a esquerda.
20. Por que Java especifica que operandos em expressões são todos avaliados da esquerda para a direita?
 21. Explique como as regras de coerção de uma linguagem afetam sua detecção de erros.